HIREHUB

Campus Placement & Internship Management System

Advanced Full-Stack BTech Final-Year Project

React.js • Java Spring Boot • MySQL • Spring Security • JWT • Docker

Project guide, implementation blueprint, source-code starter, database design, API reference, testing plan, deployment steps and GitHub workflow.

Prepared as a portfolio-ready academic project. Customize the branding, add your own screenshots and test data, and document your individual contributions before submission.

1. Project overview

HireHub is a web-based platform that brings students, recruiters and college placement administrators into one workflow. Students maintain profiles and apply for eligible jobs; recruiters publish opportunities and manage applicants; placement administrators verify companies, monitor activity and publish announcements.

Problem statement

Many colleges coordinate placements through spreadsheets, email and disconnected forms. This makes it difficult to maintain a single source of truth for student profiles, job eligibility, applications, interview stages and placement statistics. HireHub provides a role-based portal with a centralized relational database and auditable application status.

Objectives

• Create a secure, role-based web application for students, recruiters and placement administrators.

• Allow recruiters to publish internships and jobs with eligibility rules and deadlines.

• Allow students to manage profiles, view opportunities and track applications.

• Provide an administrator dashboard for company verification and placement reporting.

• Demonstrate REST API design, relational modelling, authentication, validation, testing and deployment.

Main modules

Role	Capabilities
Student	Register/login, profile, skills, academic details, browse jobs, apply, track status.
Recruiter	Company profile, create/edit openings, view applicants, shortlist/reject, update stages.
Placement Admin	Verify recruiters, manage users, moderate openings, view placement statistics.

Scope note: This guide defines a strong MVP. Before claiming production readiness, add email verification, password reset, rate limiting, audit logging, privacy review, backups and deployment monitoring.

2. Functional and non-functional requirements

Functional requirements

• Registration and login with roles STUDENT, RECRUITER and ADMIN.

• Student profile with name, department, graduation year, CGPA, skills and resume URL.

• Job/internship creation with title, description, location, minimum CGPA, eligible departments and closing date.

• Job discovery with keyword and department filters.

• Application submission with duplicate-application prevention.

• Recruiter applicant list and status updates: APPLIED, SHORTLISTED, INTERVIEW, SELECTED, REJECTED.

• Admin verification of recruiter accounts and moderation of job posts.

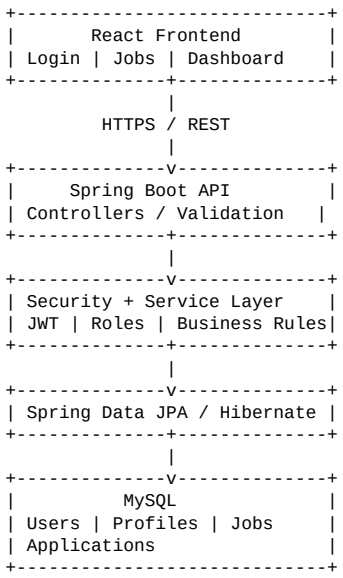
Non-functional requirements

Area	Requirement
Security	BCrypt password hashing, JWT validation, role-based endpoint access, input validation.
Performance	Pagination for job and applicant lists; indexes on common filter columns.

Area	Requirement
Reliability	Database constraints, transaction boundaries and meaningful error responses.
Usability	Responsive layout, accessible labels, clear loading and error states.
Maintainability	Layered backend, reusable React components, environment-based configuration.

3. System architecture

The browser-based React client communicates with the Spring Boot REST API over HTTPS. Spring Security validates bearer tokens and enforces roles. Services implement business rules; repositories access MySQL through Spring Data JPA.



Suggested package structure

```
hirehub/
  backend/
    src/main/java/com/hirehub/
      config/
      controller/
      dto/
      entity/
      repository/
      security/
      service/
      HireHubApplication.java
    src/main/resources/application.properties
  frontend/
    src/
      api/
      components/
      pages/
      context/
      App.jsx
      main.jsx
  docs/
    screenshots/
    API.md
  README.md
  docker-compose.yml
```

Key design decisions

• Use DTOs at the API boundary instead of exposing JPA entities directly.

• Store roles as an enum and enforce authorization on the server, not only in the UI.

• Use a unique constraint on (student_id, job_id) to prevent duplicate applications.

• Store resume files in object storage or a controlled upload service in a production version; the MVP can store a validated URL.

• Use pagination for all potentially large lists.

4. Database design

The following logical schema is sufficient for the MVP. Use migrations (Flyway or Liquibase) for repeatable schema changes.

Table	Important columns
users	id PK, name, email UNIQUE, password_hash, role, enabled, created_at
student_profiles	id PK, user_id FK UNIQUE, department, graduation_year, cgpa, skills, resume_url
recruiter_profiles	id PK, user_id FK UNIQUE, company_name, website, verified
jobs	id PK, recruiter_id FK, title, description, location, min_cgpa, deadline, active
applications	id PK, job_id FK, student_id FK, status, applied_at; UNIQUE(student_id, job_id)

Relationship summary

• One user has at most one student profile or recruiter profile, depending on role.

• One recruiter can publish many jobs.

• One student can submit many applications.

• One job can receive many applications.

• Applications connect students and jobs through a many-to-many relationship with status and timestamps.

Example MySQL schema

```
CREATE DATABASE hirehub;
USE hirehub;

CREATE TABLE users (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(120) NOT NULL,
  email VARCHAR(180) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role ENUM('STUDENT','RECRUITER','ADMIN') NOT NULL,
  enabled BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE student_profiles (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  user_id BIGINT NOT NULL UNIQUE,
  department VARCHAR(100),
  graduation_year INT,
  cgpa DECIMAL(4,2),
  skills TEXT,
  resume_url VARCHAR(500),
  CONSTRAINT fk_student_user FOREIGN KEY (user_id) REFERENCES users(id)
);

CREATE TABLE recruiter_profiles (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  user_id BIGINT NOT NULL UNIQUE,
  company_name VARCHAR(180) NOT NULL,
  website VARCHAR(300),
  verified BOOLEAN NOT NULL DEFAULT FALSE,
  FOREIGN KEY (user_id) REFERENCES users(id)
);

CREATE TABLE jobs (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  recruiter_id BIGINT NOT NULL,
  title VARCHAR(180) NOT NULL,
  description TEXT NOT NULL,
  location VARCHAR(160),
```

```
  min_cgpa DECIMAL(4,2) DEFAULT 0,
  deadline DATE NOT NULL,
  active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (recruiter_id) REFERENCES recruiter_profiles(id),
  INDEX idx_jobs_active_deadline (active, deadline)
);

CREATE TABLE applications (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  job_id BIGINT NOT NULL,
  student_id BIGINT NOT NULL,
  status ENUM('APPLIED','SHORTLISTED','INTERVIEW','SELECTED','REJECTED')
    NOT NULL DEFAULT 'APPLIED',
  applied_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (job_id) REFERENCES jobs(id),
  FOREIGN KEY (student_id) REFERENCES student_profiles(id),
  UNIQUE KEY uq_student_job (student_id, job_id)
);
```

5. Backend setup – Java Spring Boot

Recommended dependencies: Spring Web, Spring Data JPA, MySQL Driver, Spring Security, Validation, and a JWT library. Generate the project using Spring Initializr, Java 21 and Maven. Keep secrets outside source control.

application.properties

```
spring.datasource.url=${DB_URL:jdbc:mysql://localhost:3306/hirehub}
spring.datasource.username=${DB_USER:root}
spring.datasource.password=${DB_PASSWORD:change_me}
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
server.port=8080

# For a real deployment, use Flyway migrations and set ddl-auto=validate.
# Never commit production passwords or JWT signing keys.
```

Application entry point

```
package com.hirehub;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class HireHubApplication {
    public static void main(String[] args) {
        SpringApplication.run(HireHubApplication.class, args);
    }
}
```

Role enum and User entity

```
package com.hirehub.entity;

public enum Role {
    STUDENT, RECRUITER, ADMIN
}
```

```
package com.hirehub.entity;

import jakarta.persistence.*;
import java.time.Instant;

@Entity
@Table(name = "users")
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 120)
    private String name;

    @Column(nullable = false, unique = true, length = 180)
    private String email;

    @Column(name = "password_hash", nullable = false)
    private String passwordHash;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private Role role;
```

```
    private boolean enabled = true;
    private Instant createdAt = Instant.now();

    // Generate getters/setters with your IDE.
}
```

Repository and job DTO

```
package com.hirehub.repository;

import com.hirehub.entity.Job;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;

public interface JobRepository extends JpaRepository<Job, Long> {
    Page<Job> findByActiveTrue(Pageable pageable);
}
```

```
package com.hirehub.dto;

import jakarta.validation.constraints.*;
import java.math.BigDecimal;
import java.time.LocalDate;

public record CreateJobRequest(
    @NotBlank @Size(max = 180) String title,
    @NotBlank String description,
    @Size(max = 160) String location,
    @NotNull @DecimalMin("0.0") @DecimalMax("10.0") BigDecimal minCgpa,
    @NotNull @Future LocalDate deadline
) {}
```

The remaining entities (StudentProfile, RecruiterProfile, Job and Application) should follow the table schema above. Add explicit constructors, getters/setters or Lombok annotations, and JPA relationships. Keep request and response DTOs separate from persistence entities.

6. Backend API – controller and service pattern

Example job controller

```
package com.hirehub.controller;

import com.hirehub.dto.CreateJobRequest;
import com.hirehub.entity.Job;
import com.hirehub.service.JobService;
import jakarta.validation.Valid;
import org.springframework.data.domain.*;
import org.springframework.http.HttpStatus;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/jobs")
@CrossOrigin(origins = "http://localhost:5173")
public class JobController {
    private final JobService service;

    public JobController(JobService service) {
        this.service = service;
    }

    @GetMapping
    public Page<Job> list(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "10") int size
    ) {
        return service.list(PageRequest.of(page, Math.min(size, 50)));
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    @PreAuthorize("hasRole('RECRUITER')")
    public Job create(@Valid @RequestBody CreateJobRequest request) {
        return service.createForCurrentRecruiter(request);
    }
}
```

Service responsibilities

• JobService.list(Pageable): return active jobs with pagination.

• JobService.createForCurrentRecruiter(request): obtain recruiter identity from the authenticated principal, validate recruiter verification, map DTO to entity and save.

• ApplicationService.apply(jobId): verify student role, active job, deadline and eligibility; rely on database uniqueness to prevent duplicates.

• ApplicationService.updateStatus(applicationId, status): verify the authenticated recruiter owns the job before changing status.

• AdminService.verifyRecruiter(recruiterId): admin-only operation with audit information.

Important: the short controller above illustrates the layering and endpoint shape. For a complete secure implementation, return response DTOs rather than entities, derive ownership from the authenticated principal, add exception handlers, and configure the JWT filter and security rules. Do not trust a userId supplied by the browser.

7. REST API reference

Method + endpoint	Access	Purpose
POST /api/auth/register	Public	Create student or recruiter account (admin role must not be self-selecte
POST /api/auth/login	Public	Validate credentials and issue JWT.
GET /api/jobs?page=0&size=10	Authenticated	Browse active openings.
POST /api/jobs	Recruiter	Create a job.
PATCH /api/jobs/{id}	Owning recruiter	Update own job.
POST /api/jobs/{id}/apply	Student	Apply to an eligible job.
GET /api/me/applications	Student	Track own applications.
GET /api/recruiter/jobs/{id}/applications	Owning recruiter	View applicants for own job.
PATCH /api/applications/{id}/status	Owning recruiter	Update recruitment stage.
PATCH /api/admin/recruiters/{id}/verify	Admin	Verify recruiter.

Example request

```
POST /api/jobs/42/apply
Authorization: Bearer <access-token>

Response: 201 Created
{
  "applicationId": 108,
  "jobId": 42,
  "status": "APPLIED"
}
```

Standard error format

```
{
  "timestamp": "2026-11-05T12:00:00Z",
  "status": 400,
  "error": "VALIDATION_FAILED",
  "message": "The request contains invalid fields."
}
```

8. Authentication and authorization

Use Spring Security with BCryptPasswordEncoder for password hashing and a signed JWT for stateless API authentication. A token should contain a subject (user ID or email), role and expiry. Validate signature and expiry on every request.

• Registration: accept only STUDENT or RECRUITER from public requests; create ADMIN accounts through a controlled seed or admin process.

• Login: authenticate email/password and issue a short-lived access token.

• Frontend: attach the token as Authorization: Bearer .

• Backend: map roles to ROLE_STUDENT, ROLE_RECRUITER and ROLE_ADMIN.

• Ownership: check that the recruiter owns the job/application being modified.

• Production: consider refresh-token rotation, secure cookie strategy, CSRF implications, HTTPS, rate limiting and account lockout.

Security warning: Do not store plain-text passwords, expose signing secrets in React, disable authorization to make the demo work, or commit real user data. Use synthetic student records for screenshots.

9. Frontend setup – React

```
# From the project root
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm install axios react-router-dom
npm run dev
```

API client

```
// frontend/src/api/client.js
import axios from "axios";

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || "http://localhost:8080/api"
});

api.interceptors.request.use((config) => {
  const token = sessionStorage.getItem("accessToken");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

export default api;
```

Job list component

```
// frontend/src/pages/Jobs.jsx
import { useEffect, useState } from "react";
import api from "../api/client";

export default function Jobs() {
  const [jobs, setJobs] = useState([]);
  const [error, setError] = useState("");

  useEffect(() => {
    api.get("/jobs?page=0&size=10")
      .then((res) => setJobs(res.data.content ?? []))
      .catch(() => setError("Could not load jobs. Please try again."));
  }, []);

  return (
    <main className="container">
      <h1>Open opportunities</h1>
      {error && <p role="alert">{error}</p>}
      <section className="job-grid">
        {jobs.map((job) => (
          <article className="job-card" key={job.id}>
            <h2>{job.title}</h2>
            <p>{job.location}</p>
            <p>{job.description}</p>
            <small>Apply before: {job.deadline}</small>
          </article>
        ))}
      </section>
    </main>
  );
}
```

Simple responsive styling

```
/* frontend/src/index.css */
:root { font-family: Inter, system-ui, sans-serif; color: #17263c; }
body { margin: 0; background: #f4f7fb; }
.container { max-width: 1100px; margin: 0 auto; padding: 2rem; }
.job-grid { display: grid; grid-template-columns:
  repeat(auto-fit, minmax(260px, 1fr)); gap: 1rem; }
.job-card { background: white; padding: 1.2rem; border-radius: 14px;
  border: 1px solid #dce5ef; box-shadow: 0 5px 18px #1b355010; }
button { cursor: pointer; padding: .7rem 1rem; border-radius: 8px; }
```

Build Login, Register, StudentDashboard, RecruiterDashboard, AdminDashboard, JobDetails and MyApplications pages using the same API client. Protect routes for user experience, but remember that backend authorization is the real security boundary.

10. Frontend screens and user journeys

Screen	What to show
Landing / Login	Project value proposition, sign-in form, registration link and validation messages.
Student dashboard	Profile completion, recommended openings, application status summary.
Jobs page	Search, department/CGPA filters, deadline and job detail link.
Job detail	Role description, eligibility, deadline and apply action.
Recruiter dashboard	Create opening, active jobs, applicant counts and applicant pipeline.
Admin dashboard	Recruiter verification queue, job moderation and aggregate placement charts.

Suggested demo journey

• Register a student and complete the academic profile.

• Register a recruiter and have the admin verify the recruiter.

• Create a sample software developer internship.

• Log in as the student, view the job and apply.

• Log in as the recruiter, shortlist the student and move the application to INTERVIEW.

• Log in as the student and confirm the updated status appears.

11. Local development on macOS

Install Java 21, Node.js LTS, MySQL and Git. Homebrew is a convenient option if it is already installed.

```
# Check tools
java -version
node -v
npm -v
git --version

# Create database (start MySQL first)
mysql -u root -p
# Then inside MySQL:
CREATE DATABASE hirehub;
EXIT;

# Backend: from hirehub/backend
./mvnw spring-boot:run

# Frontend: in another terminal, from hirehub/frontend
npm install
npm run dev
```

Create backend environment variables in your shell or IDE run configuration. Example local values: DB_URL=jdbc:mysql://localhost:3306/hirehub, DB_USER=root and DB_PASSWORD set to your local MySQL password. Never paste a real password into a public repository.

Environment files

```
# frontend/.env.local (do not commit)
VITE_API_URL=http://localhost:8080/api

# backend environment variables (do not commit secrets)
DB_URL=jdbc:mysql://localhost:3306/hirehub
DB_USER=root
DB_PASSWORD=your_local_password
JWT_SECRET=replace_with_a_long_random_secret
```

12. Testing strategy

Test level	Examples
Unit	Eligibility rule, deadline check, duplicate-application handling, status transitions.
Integration	Repository persistence, unique constraint, controller validation, role authorization.
API	Login, invalid token, create job, apply, update status, forbidden cross-company access.
UI	Form validation, loading state, empty state, error state, responsive layout.
Security	Password hash verification, token expiry, role escalation attempts, IDOR/ownership checks.

Minimum acceptance tests

• A student cannot create a job.

• A recruiter cannot update another recruiter's job.

• An unauthenticated user cannot apply.

• A student cannot apply twice to the same job.

• An expired job cannot accept an application.

• A recruiter cannot mark an application for a job they do not own.

• An admin can verify a recruiter; a student cannot.

Use JUnit 5 and Spring Boot Test for backend tests, Mockito for isolated service tests, Postman collections for API demonstrations, and React Testing Library for frontend component tests.

13. Docker and deployment

Example Docker Compose for MySQL

```
# docker-compose.yml
services:
  mysql:
    image: mysql:8.4
    environment:
      MYSQL_DATABASE: hirehub
      MYSQL_USER: hirehub_user
      MYSQL_PASSWORD: local_dev_password
      MYSQL_ROOT_PASSWORD: local_root_password
    ports:
      - "3306:3306"
    volumes:
      - hirehub_data:/var/lib/mysql
volumes:
  hirehub_data:
```

For a real deployment, move all credentials to the hosting provider's secret manager, restrict database network access, enable TLS, use database migrations, configure CORS to the exact frontend origin and add health checks.

Deployment checklist

• Build and test backend: ./mvnw clean test package.

• Build frontend: npm run build.

• Provision managed MySQL and run migrations.

• Deploy Spring Boot API with DB_URL, DB_USER, DB_PASSWORD and JWT_SECRET configured securely.

• Deploy the frontend static build and set VITE_API_URL to the deployed API base URL.

• Configure HTTPS, CORS, logs, backups and monitoring.

• Run smoke tests for registration, login, job creation, application and status update.

14. GitHub repository and push commands

Create an empty repository on GitHub named **hirehub-placement-portal**. Do not initialize it with a README if you already have a local README.

```
# From the hirehub project root
git init
git add .
git commit -m "Initial HireHub full-stack project"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/hirehub-placement-portal.git
git push -u origin main
```

.gitignore

```
# Java / Maven
target/
*.class

# Node
node_modules/
dist/

# Environment / secrets
.env
.env.*
!.env.example
application-local.properties

# IDE / OS
.idea/
.vscode/
.DS_Store
```

Use GitHub's web interface to add a project description, topics (react, spring-boot, mysql, full-stack, campus-placement), screenshots and a clear README. Never commit .env files, database dumps containing personal data, private keys or access tokens.

15. README template

```
# HireHub – Campus Placement & Internship Portal

A full-stack placement management platform built with React,
Java Spring Boot and MySQL.

## Features
- Role-based student, recruiter and admin workflows
- Job and internship listings
- Student applications and status tracking
- Recruiter applicant management
- Admin recruiter verification

## Tech stack
React, Java 21, Spring Boot, Spring Security, MySQL,
Spring Data JPA, JWT, Docker

## Run locally
1. Create the MySQL database `hirehub`.
2. Configure backend environment variables.
3. Run the backend with `./mvnw spring-boot:run`.
4. In frontend, run `npm install` and `npm run dev`.

## API
See docs/API.md.
```

```
## Screenshots
Add screenshots of login, student dashboard, job list,
recruiter dashboard and admin dashboard.

## Security
Do not use production secrets in this repository.
Use synthetic demo data only.

## Author
Your Name | BTech CSE | Graduation Year
```

16. Project report and viva preparation

Suggested report chapters

• Introduction and problem definition

• Existing system and limitations

• Proposed system and objectives

• Requirement analysis and feasibility

• System architecture, UML and database design

• Implementation details and technology selection

• Testing strategy, test cases and results

• Deployment, limitations and future enhancements

• Conclusion and references

Future enhancements

• Resume parsing and skill extraction with explainable matching.

• Email notifications for application status and interview schedules.

• Analytics for placement rates by department and graduation year.

• Calendar integration for interview scheduling.

• Audit trail for recruiter and admin actions.

• Cloud object storage for resumes and malware scanning for uploads.

Viva questions

Question	Key point to explain
Why Spring Boot?	Convention-based Java backend, dependency injection, REST support and mature ecosystem.
Why MySQL?	Relational data, constraints, transactions and joins suit users/jobs/applications.
What is JWT?	A signed token carrying claims; the API verifies it on protected requests.
Why hash passwords?	A one-way password hash such as BCrypt reduces harm if the credential store is exposed.
How prevent duplicate applications?	Database unique constraint plus service-level handling.
How enforce role access?	Spring Security authorization at endpoint/service level, not just hidden UI controls.
How scale job search?	Indexes, pagination, query tuning, caching where appropriate and horizontal API scaling.

17. Final submission checklist

• Working backend and frontend run locally from a clean clone.

• Database schema or migration scripts included.

• README includes exact setup steps and sample environment file.

• Screenshots show all three roles using synthetic data.

• Postman collection and API documentation included.

• Unit and integration tests pass.

• No secrets, private keys or personal student data in GitHub.

• Project report includes your own diagrams, screenshots, test evidence and contribution statement.

• Be prepared to explain every major file and design decision in the interview.

Academic integrity: Treat this guide as a structured starting point. Implement, test and understand the code yourself; follow your college's rules for AI-assisted work and cite tools or references where required.

Appendix A. Suggested build order

Milestone	Deliverable
1	Create repository, Spring Boot project and React app; connect MySQL.
2	Implement users, registration, login and role authorization.
3	Implement student and recruiter profiles.
4	Implement job creation, listing, filtering and deadlines.
5	Implement applications, duplicate prevention and status tracking.
6	Build dashboards and responsive UI for all roles.
7	Add admin verification, validation, exception handling and tests.
8	Dockerize local database, deploy, document and record demo.

Appendix B. What makes this placement-ready?

The project demonstrates a realistic business workflow rather than a single-page CRUD demo. In interviews, focus on the decisions you can defend: role and ownership checks, database constraints, DTO validation, pagination, secure configuration, tests and deployment. A polished project is not just code; it is reproducible setup, clear documentation, meaningful commits and a demo you can explain.

End of project guide.